CHAPITRE 1

Architecture des ordinateurs

PTSI

2026 – 2027

Ce premier chapitre présente brièvement les principaux composants matériels d'un ordinateur (le hardware) ainsi que les principes généraux de son système d'exploitation (le software).

Ces notions concernent surtout les ordinateurs personnels, mais s'appliquent aussi à la plupart des appareils numériques comme les tablettes ou les smartphones.

PLAN DU CHAPITRE

1.1 Architecture des ordinateurs 2
- 1.1.1 Architecture de Von Neumann 2
- 1.1.2 Mémoire principale 3
- 1.1.3 Le processeur 4
- 1.1.4 Périphériques d'entrées-sorties 5

1.2 Système d'exploitation 5

1.3 Langages de programmation 7
- 1.3.1 Qu'est-ce qu'un langage de programmation ? 7
- 1.3.2 Niveau d'abstraction 7
- 1.3.3 Interprétation et compilation 7

L'informatique moderne apparaît à la fin des années 1930, portée à la fois par des mathématiciens et des ingénieurs.

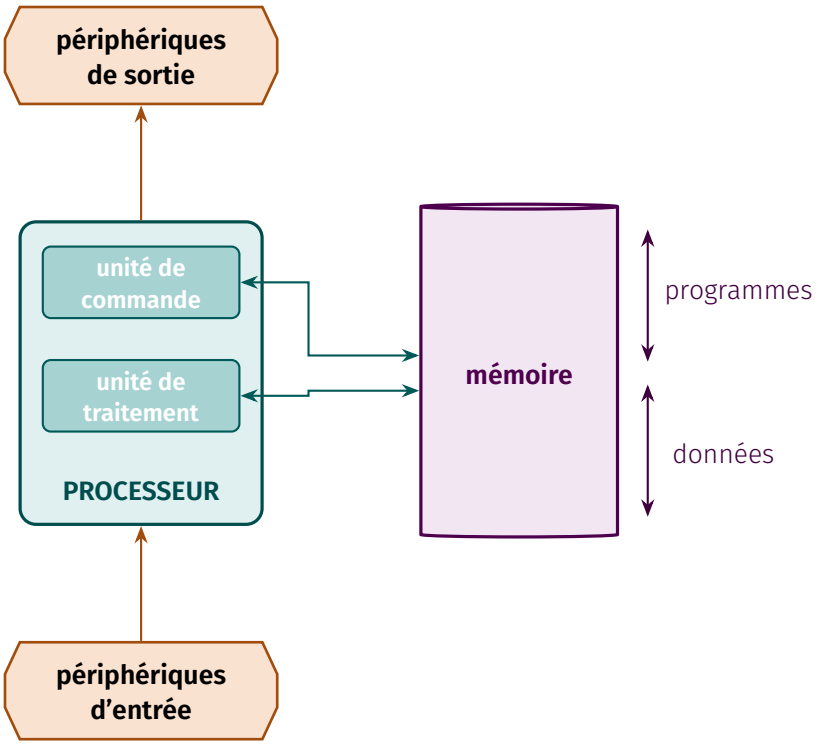
Du côté théorique, des logiciens comme Church et Turing s'interrogent sur la nature même du calcul, donnant naissance à la théorie de la calculabilité. Church développe le lambda-calcul, un formalisme fondé sur l'idée qu'une fonction peut se définir par des règles de calcul. Turing, lui, aborde le problème sous l'angle de la machine : il conçoit une machine théorique, sans existence physique à l'époque, capable de fonctionner de façon autonome. Ces deux approches, en apparence différentes, se révéleront équivalentes.

Parallèlement, des ingénieurs travaillent à la construction de machines électroniques ou électromécaniques capables de calculs rapides. Von Neumann s'impose comme figure centrale : dans le cadre du projet ENIAC, il propose dans les années 1940 une architecture de calculateur toujours d'actualité aujourd'hui. Alors que mathématiciens et ingénieurs évoluaient jusque-là de manière assez cloisonnée, ce sont ses travaux qui permettront la première concrétisation matérielle d'une machine de Turing.

1.1 Architecture des ordinateurs

1.1.1 Architecture de Von Neumann

Tous les ordinateurs reposent sur une même architecture, dite architecture de Von Neumann, proposée par le mathématicien John Von Neumann en 1948.



L'architecture d'un ordinateur selon le modèle de von Neuman.

Dans l'architecture de Von Neumann :

- La **mémoire principale** contient les données et les programmes. Le processeur y accède directement pour l'exécution de tous les programmes.
- Le **processeur** (ou micro-processeur) contient :

— une **horloge**, qui cadence toutes les opérations,
— l'**unité de commande** qui séquence l'exécution des instructions contenues dans les programmes en mémoire,
— l'**unité arithmétique et logique** qui effectue tous les calculs et les opérations logiques, et
— les **registres**, unités de mémoires d'accès ultra rapide.

1.1.2 Mémoire principale

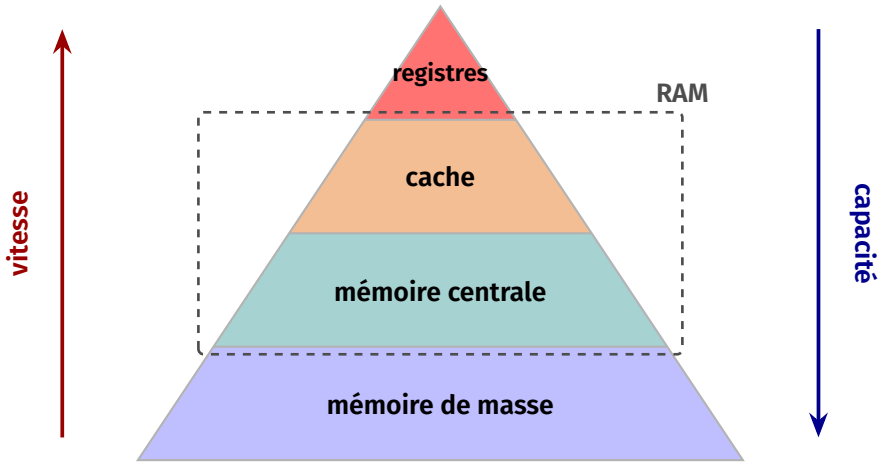
Une information manipulée par un ordinateur peut prendre des formes très diverses (texte, nombre, image, son, etc.), mais elle est toujours codée en interne sous forme binaire : une suite de 0 et de 1.

Définition 1.1

Un **bit** (de l'anglais *binary digit*, chiffre binaire) est la plus petite unité d'information manipulable par un ordinateur. Il ne peut prendre que deux valeurs : 0 ou 1.

Toute donnée traitée par la machine est donc, en dernière analyse, une suite de bits. Il convient cependant de bien distinguer un objet de sa représentation machine : deux objets de nature différente, par exemple un nombre et un caractère, peuvent partager exactement la même suite de bits en mémoire. Ce qui les distingue, c'est leur *type*, défini par le langage de programmation utilisé, ou plus simplement le contexte dans lequel ils interviennent.

Dans l'architecture de Von Neumann, la mémoire se répartit en plusieurs niveaux, du plus rapide au plus volumineux :



Les différents types de mémoire dans un ordinateur.

— les *registres*, emplacements internes au processeur utilisés pour l'exécution des instructions. Leur nombre (d'une dizaine à une centaine selon le processeur) et leur capacité (8, 16, 32 ou 64 bits chacun) restent limités ;
— la *RAM* (*Random Access Memory*, mémoire vive), qui regroupe en réalité deux couches :
  — le *cache*, mémoire tampon rapide intercalée entre le processeur et la mémoire centrale pour compenser l'écart de vitesse entre les deux. Plusieurs niveaux de cache coexistent, du plus rapide (proche des registres, capacité en kilo-octets) au plus volumineux (capacité en méga-octets) ;

- la *mémoire centrale*, qui contient le code et les données des programmes en cours d'exécution, mesurée en giga-octets. Elle est organisée en *octets* (*bytes*), chacun repéré par une *adresse*. Avec des adresses codées sur 32 ou 64 bits, on adresse au maximum 2^{32} ou 2^{64} emplacements ; or 2^{32} octets correspondent à environ 4 Go, ce qui explique le passage aux processeurs 64 bits pour dépasser cette limite ;
- la *mémoire de masse*, dispositif de stockage pérenne (disque dur, clé USB, DVD, …), accessible en lecture/écriture ou en lecture seule, et mesurée en tera-octets. Au lancement d'un programme, les données et instructions essentielles en sont copiées vers la mémoire centrale, la mémoire de masse étant nettement plus lente (un cache existe même entre les deux pour accélérer ces transferts).

Remarque

L'unité de base de la mémoire est l'*octet* (8 bits). Les préfixes usuels (kilo, méga, giga…) relèvent de la base 10, mal adaptée à un univers fonctionnant en base 2. D'où l'introduction, en 1998, de préfixes binaires dédiés (kibi, mébi, gibi…), définis par $10^3 \approx 2^{10}$. Peu employés en pratique, ces préfixes sont encore souvent confondus avec leurs équivalents décimaux : l'erreur, négligeable pour le ko (2,4 %), devient sensible pour le Go (7,4 %), voire le To (10 %).

préfixe	**valeur théorique**	**mésusage**	**préfixe**	**valeur théorique**
1 k (kilo)	$10^3 = 1\,000$	$2^{10} = 1\,024$	1 ki (kibi)	$2^{10} = 1\,024$
1 M (méga)	$10^6 = 1\,000\,000$	$2^{20} = 1\,048\,576$	1 Mi (mébi)	$2^{20} = 1\,048\,576$
1 G (giga)	$10^9 = 1\,000\,000\,000$	$2^{30} = 1\,073\,741\,824$	1 Gi (gibi)	$2^{30} = 1\,073\,741\,824$

Multiples usuels en informatique.

Lecture et écriture. Seul le processeur peut modifier l'état de la mémoire, au moyen de deux opérations : l'*écriture* (il fournit une valeur et une adresse, que la mémoire enregistre) et la *lecture* (il indique une adresse et récupère la valeur qui y est stockée).

1.1.3 Le processeur

C'est le cœur de l'ordinateur. Il est constitué de :

- L'**horloge**. Elle détermine la rapidité du processeur, et par voie de conséquence de l'ordinateur. Sa rapidité est donnée en Hz, MHz, GHz. Ainsi un processeur cadencé à 3 GHz effectue 3 milliards d'opérations à la seconde. Toutes les opérations de l'ordinateur sont cadencées par l'horloge.
- L'**unité de commande** ; elle exécute les instructions des programmes. Ces instructions sont lues séquentiellement dans la mémoire principale ; elles sont codées en langage machine. Les instructions du langage machine dépendent du microprocesseur et sont en nombre très restreint : elles consistent en quelques opérations élémentaires ; lire le contenu d'une unité de stockage située à une certaine adresse (dans la mémoire principale ou via le bus), écrire une donnée à une certaine adresse (idem), commander un calcul ou une opération logique à l'unité arithmétique et logique, passer à l'instruction suivante en mémoire, ou sauter à une instruction située à une adresse donnée en mémoire principale à condition que la valeur d'un registre donné soit non nul. Une opération est exécutée à chaque temps d'horloge.

Tout ce qui s'exécute dans l'ordinateur est commandé par l'unité de commande. C'est le donneur d'ordre. Tout programme, aussi complexe soit-il, s'obtient par l'exécution de ces quelques opérations élémentaires.

— L'**unité arithmétique et logique**. Commandée par l'unité de commande, son rôle est d'exécuter tous les calculs élémentaires, addition, soustraction, multiplication, division, et surtout toutes les opérations logiques, et, ou, non, xor, décalage, dont toutes les opérations mathématiques sont déduites. Les opérandes et résultats sont stockés dans les registres. L'unité arithmétique et logique dispose aussi d'un accès direct à la mémoire principale, et d'une connexion avec le bus.

Un ordinateur « multi-core » dispose de plusieurs processeurs qui peuvent travailler simultanément.

Une carte graphique est un périphérique de sortie qui de nos jours contient son propre processeur et sa propre mémoire principale (pour fluidifier l'affichage d'images et d'animations).

1.1.4 Périphériques d'entrées-sorties

Ils sont indispensables pour communiquer avec l'ordinateur. Ils étaient initialement réduits à des entrées et sorties sous forme de cartes perforées. Les périphériques d'entrée-sorties peuvent être, par exemple :

— Périphériques d'entrée : clavier, souris, trackpad, joystick, scanner, webcam, écran tactile…
— Périphériques de sortie : écran, vidéo-projecteur, haut-parleurs, imprimante, bras articulé, etc…
— Périphériques de stockage ; ce sont des périphériques d'entrées et sorties : disques externes, lecteur/graveur CD, DVD, Blueray, clé usb, etc…
— Périphériques de communication ; ce sont des périphériques d'entrées et de sorties : carte éthernet, carte wifi, bluetooth, clés 3G, etc…

1.2 Système d'exploitation

Le *système d'exploitation* est le premier programme exécuté lors de la mise en marche de l'ordinateur ; il permet d'accéder aux ressources matérielles de celui-ci, en particulier les organes d'entrées/sorties et la mémoire de masse (le disque dur). Le rôle principal du système d'exploitation est d'isoler les programmes des détails du matériel : un programme désirant afficher une image ne va pas envoyer directement des instructions à la carte graphique de l'ordinateur, mais plutôt demander au système d'exploitation de le faire. C'est ce dernier qui doit connaitre les détails du matériel (dans ce cas le type de carte graphique et les instructions qu'elle comprend). Cette répartition des rôles simplifie grandement l'écriture des programmes d'application.

Jean-Pierre Becirspahic

De cette première tâche découle une seconde : celle d'assurer l'interface entre le ou les utilisateurs et la machine, en fournissant à chacun d'eux une machine virtuelle à travers laquelle chacun pourra interagir avec la machine.

Définition 1.2 · Système d'exploitation

Un système d'exploitation est un ensemble de programmes permettant de gérer de façon conviviale et efficace les ressources de l'ordinateur. Il réalise un pont entre l'utilisateur et le processeur.

Actuellement, les systèmes d'exploitation les plus connus appartiennent à l'une des deux familles suivantes : la famille UNIX (Mac OS X et Linux pour les ordinateurs, iOS et Android pour les tablettes et smartphones) et la famille WINDOWS.

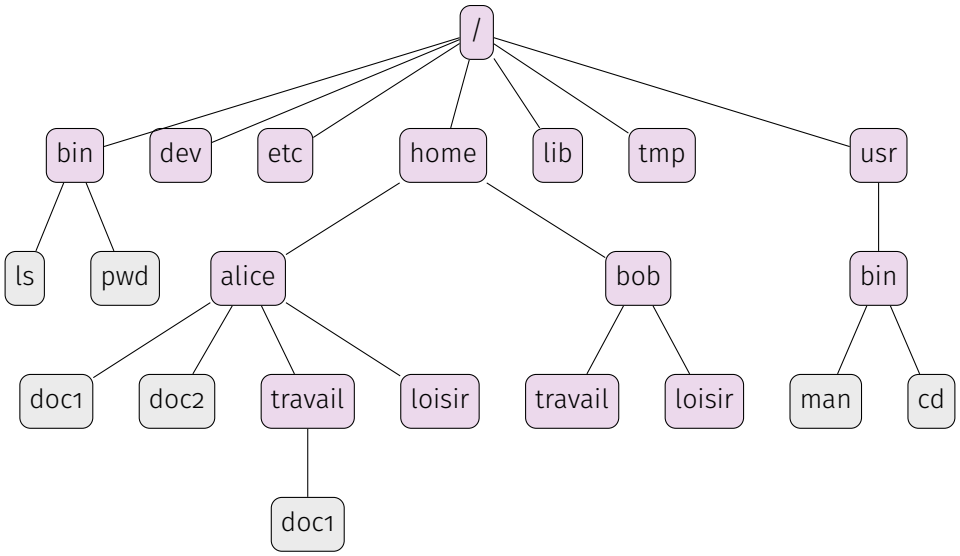
Arborescence des fichiers

Quel que soit le système d'exploitation, l'utilisateur interagit avec une machine virtuelle bien différente de la machine réelle. Dans cette machine virtuelle, l'ensemble des fichiers (applications ou données) est structuré sous forme arborescente : les feuilles de cet arbre sont les fichiers, les nœuds étant appelés des *répertoires*. On est bien loin de la machine réelle pour laquelle un fichier n'est pas nécessairement constitué d'emplacements consécutifs en mémoire (c'est le phénomène de *fragmentation*).

Sous UNIX par exemple, la racine de cet arbre est désignée par `/` ; c'est un répertoire qui contient lui-même d'autres répertoires : `bin` pour les exécutables, `dev` pour les périphériques, `etc` pour le système, `home` pour les utilisateurs, `tmp` pour les fichiers temporaires, `usr` pour les outils, etc. et ces répertoires contiennent eux-mêmes d'autres répertoires et fichiers.

Au sein de cette hiérarchie, chaque utilisateur possède une branche de l'arbre constituée d'un répertoire d'accueil qui contient les sous-répertoires et fichiers qui lui appartiennent. Dans l'exemple représenté ici, Alice et Bob possèdent tous deux un répertoire personnel rangé dans le répertoire `home`.

Chemin d'accès Tout fichier est référencé par son chemin d'accès, c'est à dire la description du chemin qu'il faut parcourir dans l'arborescence à partir d'un certain répertoire pour atteindre le fichier en question. Le chemin est spécifié par les noms des répertoires séparés par le caractère « / » et suivi du nom du fichier.



Un exemple partiel d'arborescence UNIX

Lorsqu'on commence la description du chemin par la racine, on dit que le chemin est *absolu*. Par exemple, l'un des deux documents possédé par Alice est référencé par le chemin : `/home/alice/doc1`. Il n'a bien entendu rien à voir avec le fichier `/home/bob/travail/doc1` possédé par Bob.

Il est aussi possible de décrire un fichier relativement à la position d'un autre répertoire ; on parle alors de chemin *relatif*. Sous UNIX par exemple, le symbole « ~ » désigne le répertoire d'accueil d'un utilisateur. Lors d'une session ouverte par Bob, le document qui se trouve dans le répertoire `travail` peut être décrit par : `~/travail/doc1`. Enfin, un fichier peut être décrit relativement au répertoire courant. Ce

dernier est représenté par le caractère « . », et on peut remonter dans la hiérarchie au répertoire père d'un répertoire donné en le désignant par « .. ». Par exemple, si le répertoire courant est le répertoire `loisir` de Bob, on peut décrire le premier des deux fichiers d'Alice par : `../../alice/doc1`.

1.3 Langages de programmation

1.3.1 Qu'est-ce qu'un langage de programmation ?

Le langage machine, c'est-à-dire le code binaire directement interprété par le processeur, n'est guère accessible : chaque processeur possède le sien, et y programmer directement resterait réservé à une poignée d'initiés. Un langage de programmation offre un niveau de description plus proche de l'humain : il permet, selon une syntaxe précise, d'exprimer un algorithme comme une suite d'instructions. Cette suite d'instructions est ensuite traduite en langage machine par un programme dédié – compilateur ou interpréteur. Un langage de programmation est donc un intermédiaire qui permet de piloter indirectement le cœur de l'ordinateur, sans avoir à en maîtriser les moindres détails.

Cette année, nous utiliserons le langage `Python`, dans sa troisième version.

1.3.2 Niveau d'abstraction

Les langages de programmation se distinguent d'abord par leur **niveau d'abstraction**, c'est-à-dire la distance qui les sépare du langage machine et des contraintes matérielles (gestion de la mémoire, des périphériques…) :

- un langage de **bas niveau** reste proche de la machine ; il privilégie l'efficacité, au prix d'une gestion fine et fastidieuse des ressources (Assembleur, C) ;
- un langage de **haut niveau** se rapproche du raisonnement humain et automatise la gestion des ressources, au bénéfice du confort de programmation – mais au détriment, souvent, de l'efficacité.

Ce compromis explique que les langages de haut niveau suffisent dans l'immense majorité des usages courants (calcul scientifique, applications ponctuelles…), alors que les contextes exigeants en performance ou en réactivité (cryptographie, systèmes embarqués…) imposent plutôt un langage de bas niveau. Il n'est d'ailleurs pas rare, dans l'industrie, qu'un algorithme soit d'abord prototypé dans un langage de haut niveau avant d'être retraduit en C pour l'embarquer.

`Python` se situe à l'extrême du spectre : conçu pour affranchir l'utilisateur de toute contrainte matérielle, il fournit nativement (ou via des modules) la plupart des algorithmes classiques (tri, recherche de motifs, calcul numérique…). Ce confort a une contrepartie : la maîtrise du coût réel des algorithmes ainsi appelés devient moins immédiate.

1.3.3 Interprétation et compilation

Les langages diffèrent aussi par la façon dont ils sont traduits en langage machine – une traduction qui prend en charge d'autant plus de « basse besogne » que le langage de départ est de haut niveau. On distingue deux approches :

- **la compilation** : un programme, le *compilateur*, traduit intégralement le code source en langage machine (ou en assembleur) avant toute exécution, produisant un exécutable directement utilisable par le processeur. Une fois cette étape effectuée, le programme s'exécute rapidement, ce qui convient bien à un programme achevé et appelé à être utilisé souvent. En contrepartie, la

compilation peut être lente, et exige un code syntaxiquement complet : impossible de vérifier le programme par petits morceaux ;

— **l'interprétation** : le code n'est pas converti en un autre programme, mais exécuté dynamiquement, instruction après instruction, par un *interpréteur* qui traduit et transmet chaque instruction au processeur au fur et à mesure. Ce mode permet d'exécuter un programme incomplet et d'en observer le comportement immédiatement, y compris en console, instruction par instruction – un atout précieux pour repérer les erreurs de syntaxe au moment même où elles sont écrites, ou pour exécuter des scripts hébergés à distance (PHP, JavaScript…). En contrepartie, le programme final s'exécute plus lentement, la traduction repartant toujours du code source.

`Python` adopte une approche hybride, semi-compilée et interprétée : il offre le confort d'une exécution en console et une détection dynamique des erreurs de syntaxe, mais démarre néanmoins par une phase de compilation partielle qui impose une syntaxe complète.